

Dynamic Semantics of ReOCL

Francisco Martínez-Lasaca Esther Guerra Juan de Lara

June 2026

This is a companion document for the “ReOCL: An Incremental Core of OCL for Reactive Web-Based Modeling” paper.

Contents

1	Value Domain	2
2	Auxiliary Functions	2
3	Evaluation Rules	2
3.1	Literals and Variables	2
3.2	Navigation	3
3.3	Boolean Connectives	3
3.4	Equality	4
3.5	Arithmetic	4
3.6	Comparisons	4
3.7	View-Producing Operators	5
3.8	Terminal Operators	5
3.9	Type Tests	7
3.10	Invariant Validity	7

1 Value Domain

$$v \in \text{Val} ::= \text{VTrue} \mid \text{VFalse} \mid \text{VInt}(n) \mid \text{VString}(s) \mid \text{VObj}(oid, C) \mid \text{VColl}(vs)$$

$$\llbracket e \rrbracket(\gamma, \sigma, \sigma_{\text{pre}}) \in \text{option}(\text{Val}) ::= \text{Some}(v) \mid \text{None}$$

where γ is the variable environment, σ is the current store, and σ_{pre} is the pre-state snapshot mapping locations to optional values.

2 Auxiliary Functions

$$\text{boolVal}(b) \triangleq \begin{cases} \text{VTrue}, & \text{if } b = \text{true} \\ \text{VFalse}, & \text{otherwise} \end{cases}$$

$$\text{expectBool}(v) \triangleq \begin{cases} \text{Some}(\text{true}), & \text{if } v = \text{VTrue} \\ \text{Some}(\text{false}), & \text{if } v = \text{VFalse} \\ \text{None}, & \text{otherwise} \end{cases}$$

$$\text{expectInt}(v) \triangleq \begin{cases} \text{Some}(n), & \text{if } v = \text{VInt}(n) \\ \text{None}, & \text{otherwise} \end{cases}$$

$$\text{expectObj}(v) \triangleq \begin{cases} \text{Some}(oid, C), & \text{if } v = \text{VObj}(oid, C) \\ \text{None}, & \text{otherwise} \end{cases}$$

$$\text{expectColl}(v) \triangleq \begin{cases} \text{Some}(vs), & \text{if } v = \text{VColl}(vs) \\ \text{None}, & \text{otherwise} \end{cases}$$

$$\text{readField}(\sigma, oid, C, f) \triangleq \text{Some}(\sigma(\text{fieldStatId}(C, oid, f)))$$

$$\text{readPre}(\sigma_{\text{pre}}, oid, C, f) \triangleq \sigma_{\text{pre}}(\text{fieldStatId}(C, oid, f))$$

3 Evaluation Rules

3.1 Literals and Variables

$$\frac{}{\llbracket \text{true} \rrbracket(\gamma, \sigma, \sigma_{\text{pre}}) = \text{Some}(\text{VTrue})} \text{E-TRUE}$$

$$\frac{}{\llbracket \text{false} \rrbracket(\gamma, \sigma, \sigma_{\text{pre}}) = \text{Some}(\text{VFalse})} \text{E-FALSE}$$

$$\frac{}{\llbracket n \rrbracket(\gamma, \sigma, \sigma_{\text{pre}}) = \text{Some}(\text{VInt}(n))} \text{E-INTLIT}$$

$$\frac{}{\llbracket s \rrbracket(\gamma, \sigma, \sigma_{\text{pre}}) = \text{Some}(\text{VString}(s))} \text{E-STRINGLIT} \quad \frac{}{\llbracket \text{self} \rrbracket(\gamma, \sigma, \sigma_{\text{pre}}) = \gamma(\text{self})} \text{E-SELF}$$

$$\frac{}{\llbracket x \rrbracket(\gamma, \sigma, \sigma_{\text{pre}}) = \gamma(x)} \text{E-VAR}$$

3.2 Navigation

$$\frac{\llbracket e \rrbracket(\gamma, \sigma, \sigma_{\text{pre}}) = \text{Some}(\text{VObj}(\text{oid}, C))}{\llbracket e.f \rrbracket(\gamma, \sigma, \sigma_{\text{pre}}) = \text{readField}(\sigma, \text{oid}, C, f)} \text{E-NAV}$$

$$\frac{\llbracket e \rrbracket(\gamma, \sigma, \sigma_{\text{pre}}) = \text{Some}(\text{VObj}(\text{oid}, C))}{\llbracket e.f@pre \rrbracket(\gamma, \sigma, \sigma_{\text{pre}}) = \text{readPre}(\sigma_{\text{pre}}, \text{oid}, C, f)} \text{E-PRE}$$

3.3 Boolean Connectives

$$\frac{\llbracket e_1 \rrbracket(\gamma, \sigma, \sigma_{\text{pre}}) = \text{Some}(\text{VFalse})}{\llbracket e_1 \text{ and } e_2 \rrbracket(\gamma, \sigma, \sigma_{\text{pre}}) = \text{Some}(\text{VFalse})} \text{E-AND-FALSE}$$

$$\frac{\llbracket e_1 \rrbracket(\gamma, \sigma, \sigma_{\text{pre}}) = \text{Some}(\text{VTrue}) \quad \llbracket e_2 \rrbracket(\gamma, \sigma, \sigma_{\text{pre}}) = \text{Some}(v) \quad \text{expectBool}(v) = \text{Some}(b)}{\llbracket e_1 \text{ and } e_2 \rrbracket(\gamma, \sigma, \sigma_{\text{pre}}) = \text{Some}(\text{boolVal}(b))} \text{E-AND-TRUE}$$

$$\frac{\llbracket e_1 \rrbracket(\gamma, \sigma, \sigma_{\text{pre}}) = \text{Some}(\text{VTrue})}{\llbracket e_1 \text{ or } e_2 \rrbracket(\gamma, \sigma, \sigma_{\text{pre}}) = \text{Some}(\text{VTrue})} \text{E-OR-TRUE}$$

$$\frac{\llbracket e_1 \rrbracket(\gamma, \sigma, \sigma_{\text{pre}}) = \text{Some}(\text{VFalse}) \quad \llbracket e_2 \rrbracket(\gamma, \sigma, \sigma_{\text{pre}}) = \text{Some}(v) \quad \text{expectBool}(v) = \text{Some}(b)}{\llbracket e_1 \text{ or } e_2 \rrbracket(\gamma, \sigma, \sigma_{\text{pre}}) = \text{Some}(\text{boolVal}(b))} \text{E-OR-FALSE}$$

$$\frac{\llbracket e_1 \rrbracket(\gamma, \sigma, \sigma_{\text{pre}}) = \text{Some}(\text{VFalse})}{\llbracket e_1 \text{ implies } e_2 \rrbracket(\gamma, \sigma, \sigma_{\text{pre}}) = \text{Some}(\text{VTrue})} \text{E-IMPLIES-FALSE}$$

$$\frac{\llbracket e_1 \rrbracket(\gamma, \sigma, \sigma_{\text{pre}}) = \text{Some}(\text{VTrue}) \quad \llbracket e_2 \rrbracket(\gamma, \sigma, \sigma_{\text{pre}}) = \text{Some}(v) \quad \text{expectBool}(v) = \text{Some}(b)}{\llbracket e_1 \text{ implies } e_2 \rrbracket(\gamma, \sigma, \sigma_{\text{pre}}) = \text{Some}(\text{boolVal}(b))} \text{E-IMPLIES-TRUE}$$

$$\frac{\llbracket e_1 \rrbracket(\gamma, \sigma, \sigma_{\text{pre}}) = \text{Some}(v_1) \quad \llbracket e_2 \rrbracket(\gamma, \sigma, \sigma_{\text{pre}}) = \text{Some}(v_2) \quad \text{expectBool}(v_1) = \text{Some}(b_1) \quad \text{expectBool}(v_2) = \text{Some}(b_2)}{\llbracket e_1 \text{ xor } e_2 \rrbracket(\gamma, \sigma, \sigma_{\text{pre}}) = \text{Some}(\text{boolVal}(b_1 \oplus b_2))} \text{E-XOR}$$

$$\frac{\llbracket e \rrbracket(\gamma, \sigma, \sigma_{\text{pre}}) = \text{Some}(v) \quad \text{expectBool}(v) = \text{Some}(b)}{\llbracket \text{not } e \rrbracket(\gamma, \sigma, \sigma_{\text{pre}}) = \text{Some}(\text{boolVal}(\neg b))} \text{E-NOT}$$

$$\frac{\llbracket e_1 \rrbracket(\gamma, \sigma, \sigma_{\text{pre}}) = \text{Some}(\text{VTrue}) \quad \llbracket e_2 \rrbracket(\gamma, \sigma, \sigma_{\text{pre}}) = \text{Some}(v)}{\llbracket \text{if } e_1 \text{ then } e_2 \text{ else } e_3 \text{ endif} \rrbracket(\gamma, \sigma, \sigma_{\text{pre}}) = \text{Some}(v)} \text{E-IF-TRUE}$$

$$\frac{\llbracket e_1 \rrbracket(\gamma, \sigma, \sigma_{\text{pre}}) = \text{Some}(\text{VFalse}) \quad \llbracket e_3 \rrbracket(\gamma, \sigma, \sigma_{\text{pre}}) = \text{Some}(v)}{\llbracket \text{if } e_1 \text{ then } e_2 \text{ else } e_3 \text{ endif} \rrbracket(\gamma, \sigma, \sigma_{\text{pre}}) = \text{Some}(v)} \text{E-IF-FALSE}$$

3.4 Equality

$$\frac{\llbracket e_1 \rrbracket(\gamma, \sigma, \sigma_{\text{pre}}) = \text{Some}(v_1) \quad \llbracket e_2 \rrbracket(\gamma, \sigma, \sigma_{\text{pre}}) = \text{Some}(v_2)}{\llbracket e_1 = e_2 \rrbracket(\gamma, \sigma, \sigma_{\text{pre}}) = \text{Some}(\text{boolVal}(v_1 = v_2))} \text{E-EQ}$$

$$\frac{\llbracket e_1 \rrbracket(\gamma, \sigma, \sigma_{\text{pre}}) = \text{Some}(v_1) \quad \llbracket e_2 \rrbracket(\gamma, \sigma, \sigma_{\text{pre}}) = \text{Some}(v_2)}{\llbracket e_1 <> e_2 \rrbracket(\gamma, \sigma, \sigma_{\text{pre}}) = \text{Some}(\text{boolVal}(\neg(v_1 = v_2)))} \text{E-NEQ}$$

Equality is structural: $\text{VTrue} \neq \text{VInt}(0)$, $\text{VColl}(vs_1) = \text{VColl}(vs_2)$ iff element-wise equality.

3.5 Arithmetic

$$\frac{\llbracket e_1 \rrbracket(\gamma, \sigma, \sigma_{\text{pre}}) = \text{Some}(\text{VInt}(n)) \quad \llbracket e_2 \rrbracket(\gamma, \sigma, \sigma_{\text{pre}}) = \text{Some}(\text{VInt}(m))}{\llbracket e_1 + e_2 \rrbracket(\gamma, \sigma, \sigma_{\text{pre}}) = \text{Some}(\text{VInt}(n + m))} \text{E-ADD}$$

$$\frac{\llbracket e_1 \rrbracket(\gamma, \sigma, \sigma_{\text{pre}}) = \text{Some}(\text{VInt}(n)) \quad \llbracket e_2 \rrbracket(\gamma, \sigma, \sigma_{\text{pre}}) = \text{Some}(\text{VInt}(m))}{\llbracket e_1 - e_2 \rrbracket(\gamma, \sigma, \sigma_{\text{pre}}) = \text{Some}(\text{VInt}(n - m))} \text{E-SUB}$$

$$\frac{\llbracket e_1 \rrbracket(\gamma, \sigma, \sigma_{\text{pre}}) = \text{Some}(\text{VInt}(n)) \quad \llbracket e_2 \rrbracket(\gamma, \sigma, \sigma_{\text{pre}}) = \text{Some}(\text{VInt}(m))}{\llbracket e_1 * e_2 \rrbracket(\gamma, \sigma, \sigma_{\text{pre}}) = \text{Some}(\text{VInt}(n \cdot m))} \text{E-MUL}$$

$$\frac{\llbracket e_1 \rrbracket(\gamma, \sigma, \sigma_{\text{pre}}) = \text{Some}(\text{VInt}(n)) \quad \llbracket e_2 \rrbracket(\gamma, \sigma, \sigma_{\text{pre}}) = \text{Some}(\text{VInt}(m)) \quad m \neq 0}{\llbracket e_1 / e_2 \rrbracket(\gamma, \sigma, \sigma_{\text{pre}}) = \text{Some}(\text{VInt}(n \div m))} \text{E-DIV}$$

Division by zero yields None (no rule applies).

3.6 Comparisons

$$\frac{\llbracket e_1 \rrbracket(\gamma, \sigma, \sigma_{\text{pre}}) = \text{Some}(\text{VInt}(n)) \quad \llbracket e_2 \rrbracket(\gamma, \sigma, \sigma_{\text{pre}}) = \text{Some}(\text{VInt}(m))}{\llbracket e_1 < e_2 \rrbracket(\gamma, \sigma, \sigma_{\text{pre}}) = \text{Some}(\text{boolVal}(n < m))} \text{E-LT}$$

$$\frac{\llbracket e_1 \rrbracket(\gamma, \sigma, \sigma_{\text{pre}}) = \text{Some}(\text{VInt}(n)) \quad \llbracket e_2 \rrbracket(\gamma, \sigma, \sigma_{\text{pre}}) = \text{Some}(\text{VInt}(m))}{\llbracket e_1 > e_2 \rrbracket(\gamma, \sigma, \sigma_{\text{pre}}) = \text{Some}(\text{boolVal}(n > m))} \text{E-GT}$$

$$\frac{\llbracket e_1 \rrbracket(\gamma, \sigma, \sigma_{\text{pre}}) = \text{Some}(\text{VInt}(n)) \quad \llbracket e_2 \rrbracket(\gamma, \sigma, \sigma_{\text{pre}}) = \text{Some}(\text{VInt}(m))}{\llbracket e_1 \leq e_2 \rrbracket(\gamma, \sigma, \sigma_{\text{pre}}) = \text{Some}(\text{boolVal}(n \leq m))} \text{E-LEQ}$$

$$\frac{\llbracket e_1 \rrbracket(\gamma, \sigma, \sigma_{\text{pre}}) = \text{Some}(\text{VInt}(n)) \quad \llbracket e_2 \rrbracket(\gamma, \sigma, \sigma_{\text{pre}}) = \text{Some}(\text{VInt}(m))}{\llbracket e_1 \geq e_2 \rrbracket(\gamma, \sigma, \sigma_{\text{pre}}) = \text{Some}(\text{boolVal}(n \geq m))} \text{E-GEQ}$$

3.7 View-Producing Operators

$$\frac{\begin{array}{l} \llbracket e_1 \rrbracket(\gamma, \sigma, \sigma_{\text{pre}}) = \text{Some}(\text{VColl}(vs)) \\ \forall v \in vs. \llbracket e_2 \rrbracket(\gamma[x \mapsto v], \sigma, \sigma_{\text{pre}}) = \text{Some}(r_v), \quad r_v \in \{\text{VTrue}, \text{VFalse}\} \end{array}}{\llbracket e_1 \rightarrow \text{select}(x \mid e_2) \rrbracket(\gamma, \sigma, \sigma_{\text{pre}}) = \text{Some}(\text{VColl}(\{v \in vs \mid r_v = \text{VTrue}\}))} \text{E-SELECT}$$

$$\frac{\begin{array}{l} \llbracket e_1 \rrbracket(\gamma, \sigma, \sigma_{\text{pre}}) = \text{Some}(\text{VColl}(vs)) \\ \forall v \in vs. \llbracket e_2 \rrbracket(\gamma[x \mapsto v], \sigma, \sigma_{\text{pre}}) = \text{Some}(r_v), \quad r_v \in \{\text{VTrue}, \text{VFalse}\} \end{array}}{\llbracket e_1 \rightarrow \text{reject}(x \mid e_2) \rrbracket(\gamma, \sigma, \sigma_{\text{pre}}) = \text{Some}(\text{VColl}(\{v \in vs \mid r_v = \text{VFalse}\}))} \text{E-REJECT}$$

$$\frac{\begin{array}{l} \llbracket e_1 \rrbracket(\gamma, \sigma, \sigma_{\text{pre}}) = \text{Some}(\text{VColl}(vs)) \\ \forall v \in vs. \llbracket e_2 \rrbracket(\gamma[x \mapsto v], \sigma, \sigma_{\text{pre}}) = \text{Some}(w_v) \end{array}}{\llbracket e_1 \rightarrow \text{collect}(x \mid e_2) \rrbracket(\gamma, \sigma, \sigma_{\text{pre}}) = \text{Some}(\text{VColl}(\{w_v \mid v \in vs\}))} \text{E-COLLECT}$$

3.8 Terminal Operators

$$\frac{\begin{array}{l} \llbracket e_1 \rrbracket(\gamma, \sigma, \sigma_{\text{pre}}) = \text{Some}(\text{VColl}(vs)) \\ \forall v \in vs. \llbracket e_2 \rrbracket(\gamma[x \mapsto v], \sigma, \sigma_{\text{pre}}) = \text{Some}(\text{VTrue}) \end{array}}{\llbracket e_1 \rightarrow \text{forAll}(x \mid e_2) \rrbracket(\gamma, \sigma, \sigma_{\text{pre}}) = \text{Some}(\text{VTrue})} \text{E-FORALL-TRUE}$$

$$\frac{\begin{array}{l} \llbracket e_1 \rrbracket(\gamma, \sigma, \sigma_{\text{pre}}) = \text{Some}(\text{VColl}(vs)) \\ \exists v \in vs. \llbracket e_2 \rrbracket(\gamma[x \mapsto v], \sigma, \sigma_{\text{pre}}) = \text{Some}(\text{VFalse}) \end{array}}{\llbracket e_1 \rightarrow \text{forAll}(x \mid e_2) \rrbracket(\gamma, \sigma, \sigma_{\text{pre}}) = \text{Some}(\text{VFalse})} \text{E-FORALL-FALSE}$$

ForAll may short-circuit once a false witness is found. Undefined predicate evaluations that are not needed to determine a false result are not observed.

Exists may short-circuit once a true witness is found. Undefined predicate evaluations that are not needed to determine a true result are not observed.

$$\frac{\begin{array}{l} \llbracket e_1 \rrbracket(\gamma, \sigma, \sigma_{\text{pre}}) = \text{Some}(\text{VColl}(vs)) \\ \exists v \in vs. \llbracket e_2 \rrbracket(\gamma[x \mapsto v], \sigma, \sigma_{\text{pre}}) = \text{Some}(\text{VTrue}) \end{array}}{\llbracket e_1 \rightarrow \text{exists}(x \mid e_2) \rrbracket(\gamma, \sigma, \sigma_{\text{pre}}) = \text{Some}(\text{VTrue})} \text{E-EXISTS-TRUE}$$

$$\frac{\begin{array}{l} \llbracket e_1 \rrbracket(\gamma, \sigma, \sigma_{\text{pre}}) = \text{Some}(\text{VColl}(vs)) \\ \forall v \in vs. \llbracket e_2 \rrbracket(\gamma[x \mapsto v], \sigma, \sigma_{\text{pre}}) = \text{Some}(\text{VFalse}) \end{array}}{\llbracket e_1 \rightarrow \text{exists}(x \mid e_2) \rrbracket(\gamma, \sigma, \sigma_{\text{pre}}) = \text{Some}(\text{VFalse})} \text{E-EXISTS-FALSE}$$

$$\frac{\begin{array}{l} \llbracket e_1 \rrbracket(\gamma, \sigma, \sigma_{\text{pre}}) = \text{Some}(\text{VColl}(vs)) \\ \forall v \in vs. \llbracket e_2 \rrbracket(\gamma[x \mapsto v], \sigma, \sigma_{\text{pre}}) = \text{Some}(r_v), \quad r_v \in \{\text{VTrue}, \text{VFalse}\} \\ |\{v \in vs \mid r_v = \text{VTrue}\}| = 1 \end{array}}{\llbracket e_1 \rightarrow \text{one}(x \mid e_2) \rrbracket(\gamma, \sigma, \sigma_{\text{pre}}) = \text{Some}(\text{VTrue})} \text{E-ONE}$$

$$\frac{\begin{array}{l} \llbracket e_1 \rrbracket(\gamma, \sigma, \sigma_{\text{pre}}) = \text{Some}(\text{VColl}(vs)) \\ \forall v \in vs. \llbracket e_2 \rrbracket(\gamma[x \mapsto v], \sigma, \sigma_{\text{pre}}) = \text{Some}(r_v), \quad r_v \in \{\text{VTrue}, \text{VFalse}\} \\ |\{v \in vs \mid r_v = \text{VTrue}\}| \neq 1 \end{array}}{\llbracket e_1 \rightarrow \text{one}(x \mid e_2) \rrbracket(\gamma, \sigma, \sigma_{\text{pre}}) = \text{Some}(\text{VFalse})} \text{E-ONE-FALSE}$$

$$\frac{\begin{array}{l} \llbracket e_1 \rrbracket(\gamma, \sigma, \sigma_{\text{pre}}) = \text{Some}(\text{VColl}(vs)) \\ \forall v \in vs. \llbracket e_2 \rrbracket(\gamma[x \mapsto v], \sigma, \sigma_{\text{pre}}) = \text{Some}(w_v) \quad \neg \exists i \neq j. w_i = w_j \end{array}}{\llbracket e_1 \rightarrow \text{isUnique}(x \mid e_2) \rrbracket(\gamma, \sigma, \sigma_{\text{pre}}) = \text{Some}(\text{VTrue})} \text{E-ISUNIQUE}$$

$$\frac{\begin{array}{l} \llbracket e_1 \rrbracket(\gamma, \sigma, \sigma_{\text{pre}}) = \text{Some}(\text{VColl}(vs)) \\ \forall v \in vs. \llbracket e_2 \rrbracket(\gamma[x \mapsto v], \sigma, \sigma_{\text{pre}}) = \text{Some}(w_v) \quad \exists i \neq j. w_i = w_j \end{array}}{\llbracket e_1 \rightarrow \text{isUnique}(x \mid e_2) \rrbracket(\gamma, \sigma, \sigma_{\text{pre}}) = \text{Some}(\text{VFalse})} \text{E-ISUNIQUE-FALSE}$$

$$\frac{\llbracket e \rrbracket(\gamma, \sigma, \sigma_{\text{pre}}) = \text{Some}(\text{VColl}(vs))}{\llbracket e \rightarrow \text{size}() \rrbracket(\gamma, \sigma, \sigma_{\text{pre}}) = \text{Some}(\text{VInt}(|vs|))} \text{E-SIZE}$$

$$\frac{\llbracket e \rrbracket(\gamma, \sigma, \sigma_{\text{pre}}) = \text{Some}(\text{VColl}(vs))}{\llbracket e \rightarrow \text{isEmpty}() \rrbracket(\gamma, \sigma, \sigma_{\text{pre}}) = \text{Some}(\text{boolVal}(vs = []))} \text{E-ISEMPTY}$$

$$\frac{\llbracket e \rrbracket(\gamma, \sigma, \sigma_{\text{pre}}) = \text{Some}(\text{VColl}(vs))}{\llbracket e \rightarrow \text{notEmpty}() \rrbracket(\gamma, \sigma, \sigma_{\text{pre}}) = \text{Some}(\text{boolVal}(vs \neq []))} \text{E-NOTEMPTY}$$

$$\frac{\llbracket e \rrbracket(\gamma, \sigma, \sigma_{\text{pre}}) = \text{Some}(\text{VColl}(vs)) \quad \forall v \in vs. v = \text{VInt}(n_v)}{\llbracket e \rightarrow \text{sum}() \rrbracket(\gamma, \sigma, \sigma_{\text{pre}}) = \text{Some}(\text{VInt}(\sum_{v \in vs} n_v))} \text{E-SUM}$$

Sum requires all elements to be integers; otherwise None.

3.9 Type Tests

$$\begin{array}{c}
\frac{\llbracket e \rrbracket(\gamma, \sigma, \sigma_{\text{pre}}) = \text{Some}(\text{VObj}(\text{oid}, C')) \quad C' \sqsubseteq C}{\llbracket e \rightarrow \text{ocllsKindOf}(C) \rrbracket(\gamma, \sigma, \sigma_{\text{pre}}) = \text{Some}(\text{VTrue})} \text{E-KINDOF-TRUE} \\
\\
\frac{\llbracket e \rrbracket(\gamma, \sigma, \sigma_{\text{pre}}) = \text{Some}(\text{VObj}(\text{oid}, C')) \quad C' \not\sqsubseteq C}{\llbracket e \rightarrow \text{ocllsKindOf}(C) \rrbracket(\gamma, \sigma, \sigma_{\text{pre}}) = \text{Some}(\text{VFalse})} \text{E-KINDOF-FALSE} \\
\\
\frac{\llbracket e \rrbracket(\gamma, \sigma, \sigma_{\text{pre}}) = \text{Some}(\text{VObj}(\text{oid}, C')) \quad C' = C}{\llbracket e \rightarrow \text{ocllsTypeOf}(C) \rrbracket(\gamma, \sigma, \sigma_{\text{pre}}) = \text{Some}(\text{VTrue})} \text{E-TYPEOF-TRUE} \\
\\
\frac{\llbracket e \rrbracket(\gamma, \sigma, \sigma_{\text{pre}}) = \text{Some}(\text{VObj}(\text{oid}, C')) \quad C' \neq C}{\llbracket e \rightarrow \text{ocllsTypeOf}(C) \rrbracket(\gamma, \sigma, \sigma_{\text{pre}}) = \text{Some}(\text{VFalse})} \text{E-TYPEOF-FALSE}
\end{array}$$

Both type tests require an object receiver; on non-object values they yield None.

3.10 Invariant Validity

$$\text{isTrue}(r) \Leftrightarrow r = \text{Some}(\text{VTrue})$$

$$\text{validInv}(e, \gamma, \sigma, \sigma_{\text{pre}}) \iff \text{isTrue}(\llbracket e \rrbracket(\gamma, \sigma, \sigma_{\text{pre}}))$$

$$\text{evalInvInvariant}(\text{selfOid}, \sigma, \sigma_{\text{pre}}, \text{inv}) = \llbracket \text{invBody}(\text{inv}) \rrbracket(\text{ve}, \sigma, \sigma_{\text{pre}}),$$

where $\text{ve} = [(\text{self}, \text{VObj}(\text{selfOid}, \text{invContext}(\text{inv})))]$.